

Moodle-Integrated Online Exam System

Technical Architecture, API Specification & System Rationale Report

Author / System: Technical Engineering Team

Date: June 2026

Target Stack: MERN (MongoDB, Express, React, Node)

Integration Standard: Moodle Web Services REST API

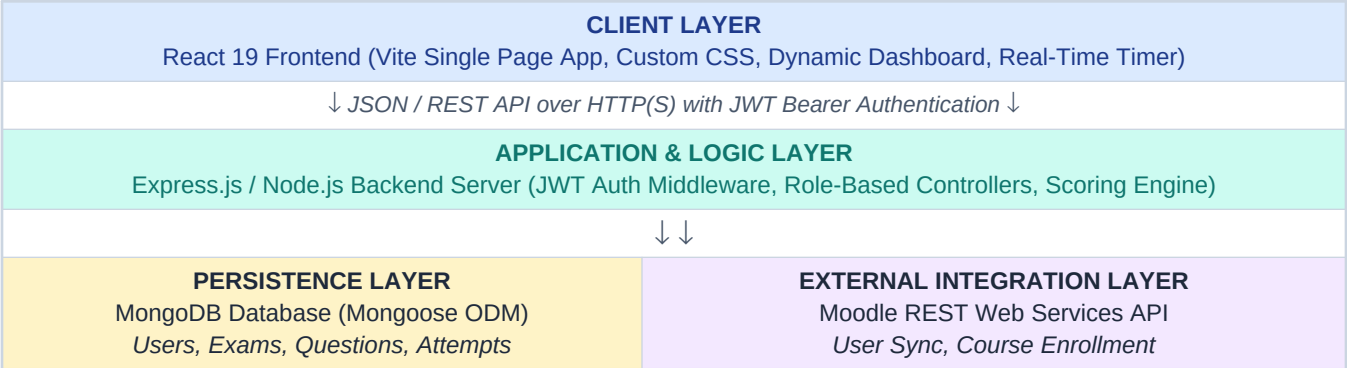
1. Executive Summary

The **Moodle-Integrated Online Exam System** is a full-stack, multi-role web platform engineered to streamline online assessment creation, dynamic student examination, automated real-time evaluation, and synchronous user/course synchronization with external Learning Management Systems (LMS), specifically Moodle. Built on the robust **MERN stack** (MongoDB, Express.js, React 19, Node.js), the platform guarantees high scalability, decoupled microservice-ready modularity, and low latency during concurrent high-stakes exam submissions.

This report presents a thorough breakdown of the platform's multi-tiered architectural layout, an exhaustive specification of all internal REST APIs and external Moodle integration endpoints, and a rigorous technical justification for every component and technology choice made.

2. System Architecture & Component Topology

The system follows a modernized **Client-Server-Database-External API** architectural pattern. By enforcing strict physical and logical separation between the user presentation layer, central business logic processing, persistent data storage, and third-party enterprise services, the application achieves fault tolerance, seamless maintainability, and independent scaling capabilities.



Detailed Layer Breakdown:

- 1. Frontend Presentation Layer (React 19 + Vite):** Serves as the interactive single-page application (SPA). It dynamically loads role-specific views (Admin, Instructor, Student), manages live client-side countdown timers, handles interactive exam rendering, and communicates asynchronously via Axios HTTP requests.
- 2. Backend Application Layer (Node.js + Express.js):** Encapsulates all domain business logic. It exposes secure RESTful endpoints, executes JWT authentication and Role-Based Access Control (RBAC), verifies submission deadlines on the server side, automatically computes grades upon submission, and formats integration payloads for Moodle.
- 3. Data Persistence Layer (MongoDB + Mongoose):** Stores system state documentations across three primary collections: Users, Exams (with embedded polymorphism for questions), and Attempts. Mongoose provides object-data modeling (ODM), enforcing type definitions, validation, and automated indexing.
- 4. Enterprise Integration Layer (Moodle Web Services API):** Wraps external HTTP calls to Moodle's REST endpoints (`server.php`). It seamlessly syncs student rosters, instructor assignments, and course offerings directly from an institutional Moodle installation.

3. Technology Stack Rationale (Why Technologies Were Chosen)

Each technology in the system architecture was selected after evaluating alternative frameworks against key operational metrics: developer velocity, performance under high concurrent loads, schema flexibility, and ease of enterprise integration.

Technology	Role in System	Technical Justification & Rationale
MongoDB & Mongoose ODM	Primary NoSQL Database	Online exams require flexible schemas because questions vary significantly (Multiple Choice, True/False, Subjective). Relational databases require rigid join tables for options and answers. MongoDB allows embedding questions and choices directly within the Exam document, drastically reducing query latency during active exams.
Express.js & Node.js	Backend Server Framework	Node's event-driven, non-blocking I/O model is uniquely suited for handling thousands of concurrent HTTP requests from students submitting answers simultaneously at the end of an exam window. Express provides lightweight routing and middleware composition for auth.
React 19 & Vite	Frontend UI Library & Bundler	React's component-driven paradigm enables instant, smooth state updates for live exam countdown timers and dynamic question cards without full-page reloads. Vite provides instantaneous Hot Module Replacement (HMR) and highly optimized production bundles.
JWT & BcryptJS	Auth & Security Infrastructure	JSON Web Tokens (JWT) enable stateless authentication, eliminating server-side session memory overhead and allowing seamless horizontal scaling. BcryptJS enforces industry-standard salted password hashing to secure stored user credentials.

Moodle Web Services REST API	External LMS Integration Client	Moodle is the world's leading open-source LMS. Utilizing Moodle's native REST web services allows educational institutions to reuse existing student rosters and course enrollments without maintaining duplicate user management infrastructure.
-------------------------------------	---------------------------------	---

4. API Specification & Specific Endpoint Analysis

The system architecture incorporates two sets of APIs: internal REST APIs designed for communication between the React client and Node backend, and external Moodle Web Service APIs for enterprise LMS integration.

4.1 Internal REST API Suite

HTTP Method & Route	Auth / Role	Purpose & Operational Rationale
POST /api/auth/login	Public	Authenticates user credentials against MongoDB, returns signed JWT token containing user ID and role.
POST /api/auth/register	Public	Registers new users, automatically hashing passwords via Bcrypt prior to insertion.
GET /health	Public	Health check probe for monitoring uptime, server timestamp, and microservice status.
GET /api/admin/users	JWT (Admin)	Retrieves all user accounts for global system auditing.
POST /api/admin/users	JWT (Admin)	Allows administrators to manually provision users with designated roles (instructor/student).
DELETE /api/admin/users/:id	JWT (Admin)	Removes a specific user from the database.
PUT /api/admin/exams/:id/stop	JWT (Admin)	Emergency override to immediately deactivate an ongoing exam across the platform.
POST /api/instructor/:id/exams	Instructor	Creates a new exam document with duration, total marks, and passing threshold.
POST /api/instructor/.../questions	Instructor	Appends structured questions (text, type, options, correct answer) into exam array.
POST /api/instructor/.../students	Instructor	Assigns list of eligible student IDs to an exam.
PUT /api/instructor/.../activate	Instructor	Publishes exam, changing state to active so eligible students can view and attempt it.
GET /api/instructor/.../attempts	Instructor	Fetches submission attempts and calculated scores for analytics and record-keeping.
POST /api/student/.../available	Student	Queries exams active for the student based on eligibility array and completion status.
POST /api/student/.../start	Student	Initializes an attempt record in MongoDB, locking in the start timestamp for timer verification.
POST /api/student/.../submit-answer	Student	Persists individual answer choice incrementally to prevent data loss on internet disconnection.
GET /api/student/.../time-remaining	Student	Returns calculated server-side remaining duration to synchronize client timer accurately.
POST /api/student/.../submit	Student	Finalizes attempt, runs automated evaluation against correct answers, and stores total score.

4.2 External Moodle Web Services Integration Client

The file backend/moodle_api/client.js implements a dedicated wrapper class, **MoodleAPIClient**, which encapsulates calls to Moodle's REST server endpoint (/webservice/rest/server.php). Moodle requires parameters encoded as application/x-www-form-urlencoded with complex array syntax. The client automatically serializes JavaScript arrays into standard Moodle URL notation (e.g. userids[0]=2).

Moodle Web Service Function	Parameters Passed	Purpose & Architectural Rationale
-----------------------------	-------------------	-----------------------------------

core_user_get_users	userids[0]=ID	Retrieves complete user profile details (fullname, email) directly from Moodle. Ensures that student identities displayed in exam transcripts match official university records exactly.
core_enrol_get_enrolled_users	courseid=ID	Queries Moodle for all students actively enrolled in a specific course module. Allows instructors to populate exam eligibility lists automatically without manual input.
core_course_get_courses	courseids[N]=ID	Fetches course metadata to associate newly created exams with recognized academic courses in Moodle.

5. Database Models & Data Modeling

The application models its domain entities in MongoDB using Mongoose schemas. The design prioritizes document embedding for high-frequency read operations during exam taking, while keeping user references lightweight.

Schema Model	Key Data Fields	Design Rationale & Sub-document Structure
User Schema (models/User.js)	username, password, role, fullname, email, userId	Stores credentials and roles (admin, instructor, student). The password field is automatically hashed using bcrypt pre-save hooks. userId links directly to Moodle ID.
Exam Schema (models/Exam.js)	exam_id, course_id, exam_name, duration, total_marks, questions[], eligible_students[]	Contains an embedded array of Question sub-documents (text, type, choices, correct answer, marks). Embedding questions inside the exam avoids relational join overhead when loading an exam for a student.
Attempt Schema (models/Attempt.js)	attempt_id, student_id, exam_id, start_time, end_time, answers, score, status	Tracks a candidate's test state. The answers field is modeled as a key-value Map (Question ID -> Selected Answer), allowing dynamic, sub-second incremental answer saving.

6. Security & Operational Workflows

Key Security & Integrity Mechanisms:

- 1. Server-Side Timer Enforcement:** Client-side timers in React are prone to tampering. When a student starts an exam, the backend locks `start_time` in MongoDB. During answer submissions or final submission, the backend independently verifies that elapsed time does not exceed `duration + grace_period`.
- 2. Automated Server-Side Grading:** Scores are calculated strictly inside `studentController.submitExam`. Answer keys are never transmitted to the frontend during an ongoing exam, preventing client-side inspection or reverse-engineering.
- 3. Role-Based Authorization Middleware:** Express endpoints are shielded by `protect` and `authorize('admin', 'instructor')` middlewares, inspecting JWT signature and roles before granting access to critical operations.

4. Moodle Resilient Fallbacks: If the local Moodle server is offline or unreachable, the `MoodleAPIClient` catches HTTP exceptions gracefully and returns mock/cached user rosters, preventing total system collapse.

Report Summary: *This architecture provides an enterprise-grade foundation combining high concurrency support with flexible assessment management and robust LMS integration.*